

Bàn về xây dựng và ứng dụng mô hình đồ thị

Huang Yuanhe

Nguồn gốc: A brief discussion of building and applying graph-theory models

I0I China candidate team papers / 2004 / Huang Yuanhe.pdf

Từ khóa

Mô hình đồ thị; xây dựng; chuyển hóa.

Tóm tắt

Trong những năm gần đây, các bài toán đồ thị xuất hiện ngày càng nhiều trong các kỳ thi tin học. Là một nhánh toán học còn khá trẻ, lý thuyết đồ thị có nhiều đặc điểm riêng so với các nhánh toán học khác. Khi dùng lý thuyết đồ thị để giải bài, ta thường đạt được lời giải hiệu quả và gọn gàng. Tuy nhiên, có công cụ này không có nghĩa là ta tự động giải được bài toán; điều quan trọng còn nằm ở khả năng nhận ra và xây dựng thành thạo một loạt mô hình đồ thị. Thông qua một số ví dụ, bài viết này giới thiệu ngắn gọn vài phương pháp mô hình hóa bằng đồ thị.

1 Mở đầu

Khi dùng kiến thức toán học để giải bài, bước đầu tiên là phân tích bài toán thực tế và xây dựng một mô hình toán học mô tả bài toán đó. Sau đó ta dùng lý thuyết và phương pháp toán học để phân tích mô hình, thu được kết quả, rồi quay lại giải quyết bài toán thực tế ban đầu. Mô hình đồ thị là một loại mô hình toán học đặc biệt. Xây dựng mô hình đồ thị nghĩa là rút ra từ nguyên mẫu của bài toán những thông tin và yếu tố hữu ích, rồi trừu tượng hóa bài toán thành quan hệ giữa đỉnh, cạnh và trọng số.

Sau khi mô hình hóa bằng đồ thị, những thông tin rời rạc và hỗn độn trở nên có quy luật; liên hệ nội tại giữa các yếu tố được thể hiện bằng quan hệ giữa đỉnh, cạnh và trọng số. Nhiều mô hình đồ thị kinh điển có thể được giải trực tiếp bằng những thuật toán xác định. Với một số bài toán phức tạp, chỉ cần nhận rõ bản chất của bài toán, nắm được điểm then chốt và xây dựng một mô hình đồ thị phù hợp, ta thường có thể chuyển chúng thành những bài toán kinh điển quen thuộc.

Điều tôi muốn trình bày trong bài viết này chính là một vài kinh nghiệm và nhận thức của bản thân về mô hình hóa bằng đồ thị.

2 Phân tích ví dụ

2.1 Ví dụ 1: Place the Robots (ZOJ)

Tóm tắt đề bài

Cho một bàn cờ $N \times M$ với $N, M \leq 50$. Mỗi ô thuộc một trong ba loại: đất trống, cỏ, hoặc tường. Robot chỉ được đặt trên đất trống. Nếu hai robot cùng hàng hoặc cùng cột và giữa chúng không có tường, chúng có thể tấn công lẫn nhau. Hỏi với bàn cờ đã cho, có thể đặt nhiều nhất bao nhiêu robot sao cho không robot nào tấn công robot nào?

Phân tích

Trong nguyên mẫu của bài toán, các thông tin như cỏ và tường không phải là điều ta quan tâm trực tiếp; ta chỉ quan tâm đến các ô trống và quan hệ giữa chúng. Vì vậy, một mô hình đơn giản rất tự nhiên là: lấy mỗi ô trống làm một đỉnh, nối cạnh giữa hai ô trống xung đột với nhau. Khi đó bài toán chuyển thành tìm tập độc lập lớn nhất của đồ thị. Như đã biết, đây là một bài toán NP-đầy đủ. Rõ ràng mô hình này không đem lại thuận lợi nào cho việc giải bài, vì vậy ta phải xây dựng một mô hình mới thực sự hiệu quả.

Ta gọi một đoạn liên tiếp trong cùng hàng hoặc cùng cột, bị ngăn cách bởi tường và có chứa ô trống, là một **khối**. Hiển nhiên trong mỗi khối nhiều nhất chỉ đặt được một robot. Ta đánh số các khối theo hàng và theo cột riêng biệt. Cụ thể, khối ngang sẽ thuộc phía X , khối dọc sẽ thuộc phía Y . Nếu hai khối có chung một ô trống, ta nối một cạnh giữa chúng. Như vậy ta thu được một đồ thị hai phía.

Trong mô hình này, mỗi cạnh biểu diễn một ô trống. Hai ô trống xung đột chắc chắn tương ứng với hai cạnh có chung một đầu mút, vì chúng nằm trong cùng một khối ngang hoặc cùng một khối dọc. Do đó, bài toán chuyển thành bài toán tìm matching lớn nhất trên đồ thị hai phía. Đây là bài toán kinh điển trong lý thuyết đồ thị và có thể giải bằng thuật toán Hungary.

Tiểu kết

So sánh hai mô hình trên, mô hình thứ nhất quá đơn giản và không nhận ra bản chất của bài toán; mô hình thứ hai nắm được đầy đủ liên hệ nội tại của bài toán và xây dựng khéo léo một mô hình đồ thị hai phía. Vì sao lại có hai kết quả khác hẳn nhau như vậy? Thứ nhất, góc nhìn phân tích bài toán khác nhau: mô hình thứ nhất lấy ô trống làm đỉnh, còn mô hình thứ hai lấy ô trống làm cạnh. Thứ hai, mô hình thứ nhất chọn chưa đủ thông tin từ nguyên mẫu, khiến mô hình được xây dựng không giữ lại được tính chất quan trọng của nguyên mẫu; trong khi đó mô hình thứ hai giữ được tính chất “bàn cờ” quan trọng. Từ đây có thể thấy việc lựa chọn thông tin là một bước cực kỳ quan trọng trong mô hình hóa bằng đồ thị.

2.2 Ví dụ 2: Thuê thu ngân (ACM Tehran 2000)

Mô tả bài toán

Một siêu thị ở Tehran mở cửa 24 giờ mỗi ngày và cần một nhóm thu ngân để đáp ứng nhu cầu phục vụ. Quản lý siêu thị thuê bạn giúp giải quyết bài toán này: ở những thời đoạn khác nhau trong ngày, siêu thị cần số lượng thu ngân khác nhau, chẳng hạn lúc nửa đêm chỉ cần một nhóm nhỏ còn buổi chiều thì cần rất nhiều người. Mục tiêu của ông ta là thuê ít thu ngân nhất.

Quản lý đã cung cấp số thu ngân tối thiểu cần có trong từng giờ của một ngày: $R_0, R_1, \dots, R_{23}$. Trong đó R_0 là số thu ngân tối thiểu cần có từ nửa đêm đến 1 giờ sáng, R_1 là số cần có từ 1 giờ đến 2 giờ sáng, v.v. Các dữ liệu này giống nhau ở mọi ngày. Có N người xin việc. Mỗi ứng viên I trong 24 giờ sẽ bắt đầu

làm từ một thời điểm cố định và làm liên tục đúng 8 giờ. Ký hiệu t_I với $0 \leq t_I \leq 23$ là thời điểm bắt đầu đó. Nghĩa là nếu ứng viên thứ I được nhận, người đó sẽ làm liên tục 8 giờ kể từ thời điểm t_I .

Bạn cần viết chương trình, nhập R_I với $I = 0, \dots, 23$ và t_I với $I = 1, \dots, N$, tất cả đều là số nguyên không âm, rồi tính số thu ngân ít nhất cần thuê để thỏa mãn các ràng buộc trên. Ở mỗi thời điểm, số thu ngân đang làm việc được phép lớn hơn giá trị R_I tương ứng.

Input. Dòng đầu của input là số bộ dữ liệu kiểm tra, không quá 20. Trong mỗi bộ, dòng đầu gồm 24 số nguyên biểu diễn $R_0, R_1, \dots, R_{23}$ với $R_I \leq 1000$. Dòng tiếp theo là N , số ứng viên, với $0 \leq N \leq 1000$. Mỗi dòng tiếp theo chứa một số nguyên t_I với $0 \leq t_I \leq 23$. Không có dòng trống giữa hai bộ dữ liệu.

Output. Với mỗi bộ dữ liệu, xuất đúng một dòng chứa một số nguyên, là số thu ngân ít nhất cần thuê. Nếu vô nghiệm, hãy xuất No Solution.

Phân tích

Thoạt nhìn bài này rất dễ khiến ta nghĩ đến tham lam, quy hoạch động hoặc mạng luồng, nhưng các thuật toán đó đều không xử lý được bài toán này một cách trực tiếp.

Vì bài toán có nhiều ràng buộc, để làm rõ ý tưởng ta trước hết biểu diễn các ràng buộc bằng ngôn ngữ toán học. Gọi $S[i]$ là tổng số thu ngân được thuê trong các thời điểm từ 0 đến i , và W_i là số ứng viên bắt đầu làm việc tại thời điểm i . Khi đó các ràng buộc trong đề bài có thể chuyển thành hệ bất đẳng thức sau:

$$\begin{cases} 0 \leq S[i] - S[i-1] \leq W_i, & 0 \leq i \leq 23, \\ S[i] - S[i-8] \leq R_i, & 8 \leq i \leq 23, \\ S[23] + S[i] - S[i+16] \leq R_i, & 0 \leq i \leq 7. \end{cases}$$

Hệ bất đẳng thức như vậy khiến ta nghĩ ngay đến hệ ràng buộc sai phân. Với mỗi bất đẳng thức $S[i] - S[j] \leq K$, ta thêm một cung có hướng từ đỉnh j đến đỉnh i với trọng số K . Giá trị nhỏ nhất của $S[23]$ mà ta cần tìm sẽ là độ dài đường đi ngắn nhất từ đỉnh 0 đến đỉnh 23. Tuy nhiên, cần chú ý bất đẳng thức thứ ba ở trên: nó chứa ba ẩn, nên không thể biểu diễn trong đồ thị như một quan hệ cạnh.

Đến đây tưởng như ta đã rơi vào bế tắc. Phải chăng bài này không thể giải bằng hệ ràng buộc sai phân? Không, ta còn cần thêm một bước chuyển hóa. Nếu xem $S[23]$ là ẩn thì chắc chắn không làm tiếp được. Nhưng nếu xem $S[23]$ là một giá trị đã biết, bất đẳng thức thứ ba chỉ còn hai ẩn $S[i]$ và $S[i+16]$; khi đó ta thêm cung từ đỉnh $i+16$ đến đỉnh i với $0 \leq i \leq 7$ và trọng số $R_i - S[23]$. Như vậy, toàn bộ hệ bất đẳng thức có thể chuyển thành một đồ thị có hướng. Nghiệm của ẩn $S[i]$ chính là đường đi ngắn nhất từ đỉnh 0 đến đỉnh i trong đồ thị. Khi trong đồ thị tồn tại chu trình âm, hệ bất đẳng thức vô nghiệm.

Cách làm trên xem $S[23]$ là giá trị đã biết, nhưng trên thực tế $S[23]$ không những là ẩn mà còn chính là giá trị ta cần tối ưu. Ta xử lý bằng cách dùng tìm kiếm nhị phân để thử giá trị của $S[23]$, dần thu hẹp phạm vi, và dùng phương pháp lập để kiểm tra có tồn tại chu trình âm hay không, tức kiểm tra tính khả thi. Nếu ngay cả khi $S[23] = N$ vẫn không khả thi, ta xuất No Solution; ngược lại ta xuất giá trị nhỏ nhất của $S[23]$. Độ phức tạp thời gian là

$$O(24^3 \log_2 N).$$

Tiểu kết

Bài này dùng lý thuyết hệ ràng buộc sai phân. Trong thi đấu, những hệ như vậy không quá thường gặp, nhưng chúng có thể giải quyết khéo léo một số bài khó. Mô hình của loại bài này thường không lộ rõ,

cần có suy nghĩ và chuyển hóa. Điểm then chốt khi làm dạng bài này là biểu diễn các ràng buộc trong đề bài thành bất đẳng thức, rồi chuyển các bất đẳng thức đó thành mô hình đường đi ngắn nhất hoặc dài nhất trên đồ thị.

2.3 Ví dụ 3: Greedy Island (ZOJ)

Tóm tắt đề bài

Có N thẻ, với $N \leq 100000$. Mỗi thẻ có ba năng lực, giá trị lần lượt là A_i, B_i, C_i . Mỗi thẻ có thể dùng một trong ba năng lực và mỗi thẻ chỉ được dùng một lần. Ta cần chọn A thẻ dùng năng lực thứ nhất, B thẻ dùng năng lực thứ hai, và C thẻ dùng năng lực thứ ba, với $A + B + C \leq 100$. Hãy tính cần dùng những thẻ nào và dùng năng lực nào của từng thẻ để tổng giá trị năng lực tương ứng là lớn nhất.

Phân tích

Có nhiều cách giải bài toán tối ưu hóa, chẳng hạn quy hoạch động hoặc mạng luồng. Bài này là một mô hình mạng luồng khá rõ.

Trong mô hình mạng luồng có nhiều loại trọng số: lưu lượng, sức chứa, và cũng có thể có chi phí. Trong bài này, sức chứa có thể dùng làm ràng buộc lựa chọn để bảo đảm tính hợp lệ của nghiệm; chi phí biểu diễn giá trị của lựa chọn để bảo đảm tính tối ưu. Vì vậy, nói chính xác hơn, bài này là một mô hình luồng cực đại chi phí cực đại.

Sau khi nhận rõ mô hình của bài toán, bước tiếp theo là dựng đồ thị:

- Mỗi thẻ i được biểu diễn bằng một đỉnh i . Thêm ba đỉnh P_1, P_2, P_3 biểu diễn ba loại năng lực, cùng với nguồn S và đích T .
- Từ nguồn lần lượt nối cung đến P_1, P_2, P_3 , có sức chứa tương ứng là A, B, C và chi phí bằng 0.
- Từ P_1, P_2, P_3 lần lượt nối cung đến mỗi đỉnh i với $1 \leq i \leq N$, sức chứa bằng 1, chi phí lần lượt là A_i, B_i, C_i .
- Từ mỗi đỉnh i với $1 \leq i \leq N$ nối một cung đến đích, sức chứa bằng 1 và chi phí bằng 0.

Nói cách khác, mạng có dạng

$$S \rightarrow \{P_1, P_2, P_3\} \rightarrow \{1, 2, \dots, N\} \rightarrow T,$$

trong đó các cung từ S đến ba đỉnh năng lực quyết định số thẻ cần chọn cho từng loại, còn các cung từ đỉnh năng lực đến thẻ mang giá trị thu được khi dùng thẻ đó cho năng lực tương ứng.

Sau khi dựng đồ thị, ta tìm luồng cực đại chi phí cực đại từ S đến T , rồi kiểm tra các cung chảy ra từ P_1, P_2, P_3 để xuất phương án tối ưu. Thuật toán này còn rất thô: độ phức tạp thời gian là $O(N^3)$, độ phức tạp bộ nhớ là $O(N^2)$, nên cả thời gian lẫn bộ nhớ đều không khả thi và cần tiếp tục tối ưu.

Bài này có đến một trăm nghìn thẻ, trong khi số thẻ cuối cùng cần chọn không vượt quá 100. Nếu trước khi dựng đồ thị ta xóa bớt các thẻ vô dụng, hiệu quả chắc chắn sẽ tăng đáng kể.

Những thẻ nào là vô dụng?

Trước hết xét việc chọn năng lực thứ nhất. Nếu sắp xếp toàn bộ thẻ theo giá trị năng lực thứ nhất giảm dần, hiển nhiên ta nên cố gắng chọn A thẻ ở phía đầu. Vì mỗi thẻ chỉ được dùng một lần, chúng có thể xung đột với hai loại năng lực còn lại; số thẻ xung đột nhiều nhất là $B + C$. Do đó các thẻ thực sự có ích đối với năng lực thứ nhất chỉ là $A + B + C$ thẻ đầu tiên.

Tương tự, với năng lực thứ hai và thứ ba, ta cũng chỉ cần giữ lại $A + B + C$ thẻ có giá trị lớn nhất của từng năng lực. Bước này có thể thực hiện trong thời gian tuyến tính.

Đây là một phương pháp vừa đơn giản vừa hiệu quả. Sau bước xử lý này, số đỉnh được giữ lại không vượt quá $3(A + B + C)$, số đỉnh giảm mạnh, và thời gian tìm luồng cực đại chi phí cực đại giảm xuống còn

$$O((A + B + C)^3).$$

Đến đây thuật toán đã được tối ưu đến mức chấp nhận được, với độ phức tạp tổng thể chỉ là

$$O(N + (A + B + C)^3).$$

Nếu muốn tăng hiệu quả hơn nữa, có thể dùng thuật toán tốt hơn để xóa các đỉnh dư thừa. Tuy nhiên điều đó không có nhiều ý nghĩa và cũng không phải trọng tâm thảo luận của bài viết này.

Ngoài ra, bài này còn có thể chuyển thành mô hình đồ thị hai phía và giải bằng thuật toán matching tối ưu. Bước này xin để độc giả tự suy nghĩ.

Tiểu kết

Bài này xây dựng mô hình mạng luồng. Hệ số trong các thuật toán thuộc lớp mô hình này thường lớn, độ phức tạp lập trình cũng cao, nên trong thi đấu nó thường được xem như một “thuật toán dự phòng” khi không còn hướng đi nào khác. Tuy vậy, mô hình mạng luồng có phạm vi áp dụng rộng. Với một số bài toán khá phức tạp hoặc có nhiều ràng buộc, mô hình mạng luồng có thể xử lý rất tốt; hơn nữa những bài dựa trên mô hình mạng luồng thường tương đối dễ nhận ra. Vì thế mô hình mạng luồng có ứng dụng rất rộng rãi.

3 Tổng kết

Bài toán biến hóa muôn hình vạn trạng, nên không có một quy tắc chung nào cho việc xây dựng mô hình đồ thị của bài toán. Các ví dụ phía trên đều tương đối đơn giản. Trong những bài toán phức tạp hơn, đôi khi ta cảm thấy khó xây dựng một mô hình thích hợp. Khi đó, với tiền đề không làm thay đổi tính chất của nguyên mẫu bài toán, ta cần trừu tượng hóa và đơn giản hóa nguyên mẫu, rồi trên cơ sở đó xây dựng một mô hình phù hợp và hiệu quả. Đôi khi sau khi đã xây dựng được một mô hình cho bài toán, ta lại cảm thấy mô hình đó khó giải. Khi ấy ta có thể cần sửa đổi hoặc chuyển hóa mô hình, hoặc phân tích sâu hơn nguyên mẫu bài toán, rút ra những yếu tố then chốt hơn để xây dựng một mô hình dễ giải. Tất cả những việc này đều đòi hỏi kinh nghiệm phong phú, tư duy linh hoạt và khả năng sáng tạo tốt.

Tài liệu tham khảo

1. Wu Wenhui, Wang Jiande, *Phân tích thuật toán thực dụng và thiết kế chương trình*, Nhà xuất bản Công nghiệp Điện tử, 1998.
2. Wu Wenhui, Wang Jiande, *Hướng dẫn thi Olympic Tin học quốc tế và toàn quốc cho thanh thiếu niên: thuật toán và thiết kế chương trình trong lý thuyết đồ thị*, Nhà xuất bản Đại học Thanh Hoa, 1997.
3. Bản thảo thảo luận giai đoạn 3 của đội tuyển tập huấn quốc gia IOI 2003.

A Nguyên đề ví dụ 1

Nguồn đề: ZOJ Monthly, October 2003 Contest.

Place the Robots

Robert là một kỹ sư nổi tiếng. Một ngày nọ, anh được sắp giao một nhiệm vụ. Bối cảnh của nhiệm vụ như sau.

Cho một bản đồ gồm các ô vuông. Có ba loại ô: tường, cỏ và ô trống. Sếp của Robert muốn đặt nhiều robot nhất có thể trên bản đồ. Mỗi robot cầm một vũ khí laser có thể bắn đồng thời theo bốn hướng bắc, đông, nam và tây. Robot phải đứng yên tại ô được đặt ban đầu và luôn bắn. Tia laser chắc chắn có thể đi qua ô cỏ, nhưng không thể đi qua ô tường. Robot chỉ được đặt trên ô trống. Dĩ nhiên sếp không muốn thấy robot này làm hại robot khác. Nói cách khác, hai robot không được nằm trên cùng một hàng hoặc cùng một cột, trừ khi giữa chúng có tường.

Vì bạn là một lập trình viên thông minh và là một trong những người bạn tốt nhất của Robert, anh ấy nhờ bạn giúp giải bài toán này: với mô tả của bản đồ, hãy tính số robot lớn nhất có thể đặt trên bản đồ.

Input. Dòng đầu chứa một số nguyên T với $T \leq 11$, là số bộ dữ liệu.

Với mỗi bộ dữ liệu, dòng đầu chứa hai số nguyên m, n với $1 \leq m, n \leq 50$, là số hàng và số cột của bản đồ. Sau đó là m dòng, mỗi dòng chứa n ký tự #, *, hoặc o, lần lượt biểu diễn tường, cỏ và ô trống.

Output. Với mỗi bộ dữ liệu, trước hết xuất số thứ tự bộ dữ liệu trên một dòng theo dạng Case :id, trong đó id là số thứ tự tính từ 1. Dòng thứ hai chỉ xuất số robot lớn nhất có thể đặt trên bản đồ đó.

Sample Input.

```
2
4 4
o***
*###
oo#o
***o
4 4
#ooo
o#oo
oo#o
***#
```

Sample Output.

```
Case :1
3
Case :2
5
```

B Nguyên đề ví dụ 3

Đề có chỉnh sửa. Nguồn đề: ZOJ Sunny Cup 2003 Online Contest.

Greedy Island

Gon và Killua đang tham gia một trò chơi tên là Greedy Island. Mục tiêu của trò chơi là thu thập 100 thẻ phép thuật rải rác khắp trò chơi. Mỗi thẻ phép thuật có ba thuộc tính: tấn công, phòng thủ và đặc biệt. Giá trị số của mỗi thuộc tính nằm trong khoảng từ 0 đến 100. Mỗi thẻ chỉ được dùng một lần. Tất cả thẻ phép thuật phải được lưu trong Book, vật phẩm ban đầu của trò chơi. Book có thể chứa nhiều nhất 50 thẻ phép thuật, nên Gon và Killua tổng cộng có thể có nhiều nhất 100 thẻ phép thuật. Hiện tại Gon và Killua có n thẻ phép thuật, và họ muốn dùng A thẻ cho tấn công, B thẻ cho phòng thủ, và C thẻ cho mục đích đặc biệt, với $A + B + C \leq 100$. Nếu $n > A + B + C$, họ phải bỏ đi một số thẻ.

Họ muốn biết tổng lớn nhất có thể của giá trị tấn công trong nhóm tấn công, giá trị phòng thủ trong nhóm phòng thủ và giá trị đặc biệt trong nhóm đặc biệt. Nếu có nhiều phương án, chọn phương án có tổng tất cả thuộc tính của tất cả các thẻ được chọn là lớn nhất.

Input. Dòng đầu chứa một số nguyên T với $1 \leq T \leq 10$, là số bộ dữ liệu.

Với mỗi bộ dữ liệu, dòng đầu chứa một số nguyên n với $n \leq 100100$; dòng tiếp theo chứa ba số nguyên A, B, C với $A, B, C \geq 0$ và $A + B + C \leq n$; tiếp đó là n dòng, mỗi dòng chứa giá trị tấn công, phòng thủ và đặc biệt của một thẻ phép thuật.

Output. Với mỗi bộ dữ liệu, in trên một dòng hai số, cách nhau bởi một dấu cách: tổng lớn nhất của giá trị tấn công trong nhóm tấn công, giá trị phòng thủ trong nhóm phòng thủ và giá trị đặc biệt trong nhóm đặc biệt; và tổng lớn nhất của tất cả thuộc tính của tất cả các thẻ.

Sample Input.

```
2
3
1 1 1
100 0 0
0 100 0
0 0 100
4
1 1 1
12 32 44
33 48 37
37 38 33
46 79 78
```

Sample Output.

```
300 300
163 429
```